

持久化

一、简介

`brick-core-orm-plus` 技术组件，基于 Mybatis Plus 实现数据库的操作，并在此基础上扩展了四个功能：

- 提供了一套注解支持联表查询功能，并对 `BaseMapper` 接口进行了扩展，增加了联表查询方法和其他常用的基础方法，同时增加了联表查询优化插件避免不必要的 Join。
- 提供了一套注解支持主从操作的功能，并增加了 `BaseService` 接口来提供主从操作方法
- 基于 [字段类型处理器](#) 实现了数据加密存储功能
- 基于拦截器实现了数据变更日志的构建

如果你没有学习过 Mybatis Plus，建议先阅读 [《Mybatis-Plus 入门》](#) 文档。

组件内置了 Mybatis Plus 的以下配置：

```
1 ## 统一指定逻辑删除属性为 dataStatus
2 mybatis-plus.global-config.db-config.logic-delete-field=dataStatus
3 ## 已被删除记录的 data_status 字段会被自动设置为与主键字段一致的值
4 mybatis-plus.global-config.db-config.logic-delete-value=#id
5 ## 未被删除记录的 data_status 字段应该设置成 1
6 mybatis-plus.global-config.db-config.logic-not-delete-value=1
7 ## 如果字段值为空，也会被查询出来
8 mybatis-plus.configuration.jdbc-type-for-null=null
```

二、持久化映射类

在项目中我们通常会将持久化映射类中的通用字段统一提取到父类中来方便我们后续的开发和管理，下面是一个 BaseDO 示例：

```
1 public abstract class BaseDO implements DataObject {
2     //创建人
3     @TableField(fill = FieldFill.INSERT)
4     private String creatorId;
5     //创建时间
6     @TableField(fill = FieldFill.INSERT)
```

```

7 private LocalDateTime createTime;
8 //末次操作人
9 @TableField(fill = FieldFill.INSERT_UPDATE)
10 private String modifierId;
11 //末次操作时间
12 @TableField(fill = FieldFill.INSERT_UPDATE)
13 private LocalDateTime alterTime;
14 //版本号
15 @Version
16 private Integer version;
17 //逻辑删除
18 private String dataStatus;
19
20 // getter and setter...
21 }

```

2.1 DataObject

这个基类实现了 `DataObject` 接口，该类与 `Serializable Cloneable RandomAccess` 接口类似只是用来标记某个类的实例是持久化数据对象。

2.2 自动填充字段值

属性 `creatorId createTime modifierId alterTime` 都使用了 `TableField` 注解指定了 [字段自动填充策略](#) 为新增或更新时都自动填充该字段的值，同时我们需要创建 `MetaObjectHandler` 的实现来填充这些字段，并作为 Spring Bean：

```

1 @Component
2 public class BaseMetaObjectHandler implements MetaObjectHandler {
3
4     private final CurrentUserInfoService currentUserInfoService;
5
6     private final String CREATOR_ID = "creatorId";
7     private final String MODIFIER_ID = "modifierId";
8     private final String CREATE_TIME = "createTime";
9     private final String ALTER_TIME = "alterTime";
10
11     public BaseMetaObjectHandler(CurrentUserInfoService currentUserInfoService) {
12         this.currentUserInfoService = currentUserInfoService;
13     }
14
15     @Override
16     public void insertFill(MetaObject metaObject) {
17         CurrentUserInfo currentUserInfo = currentUserInfoService.getCurrentUserInfo();
18         if (currentUserInfo != null) {
19             Serializable userId = currentUserInfo.getId();

```

```

20     this.setFieldValByName(CREATOR_ID, userId, metaObject);
21     this.setFieldValByName(MODIFIER_ID, userId, metaObject);
22 }
23 if (metaObject.findProperty(CREATE_TIME,true) != null) {
24     if (metaObject.getValue(CREATE_TIME) == null) {
25         this.strictInsertFill(metaObject, CREATE_TIME, LocalDateTime::now,
LocalDateTime.class);
26     }
27 }
28 if (metaObject.findProperty(ALTER_TIME,true) != null){
29     if (metaObject.getValue(ALTER_TIME) == null) {
30         this.strictInsertFill(metaObject, ALTER_TIME, LocalDateTime::now,
LocalDateTime.class);
31     }
32 }
33 }
34
35 @Override
36 public void updateFill(MetaObject metaObject) {
37     CurrentUserInfo currentUserInfo = currentUserInfoService.getCurrentUserInfo();
38     if (currentUserInfo != null) {
39         Serializable userId = currentUserInfo.getId();
40         this.setFieldValByName(MODIFIER_ID, userId, metaObject);
41     }
42     if (metaObject.findProperty(ALTER_TIME,true) != null){
43         if (metaObject.getValue(ALTER_TIME) == null) {
44             this.strictUpdateFill(metaObject, ALTER_TIME, LocalDateTime::now,
LocalDateTime.class);
45         }
46     }
47 }
48 }

```

2.3 乐观锁

属性 `version` 使用了 `Version` 注解，当要更新一条记录时希望这条记录没有被其他人更新，那么可以基于该字段来实现乐观锁：

1. 取出记录时获取当前版本号
2. 更新时带上这个版本号条件并设置新的版本号 `set version = newVersion where version = oldVersion`
3. 如果版本号不正确，就更新失败

2.4 逻辑删除

`brick-core-orm-plus` 组件内置了逻辑删除字段为 `dataStatus`，并内置了已被删除和未被删除的值；在执行查询或更新时会自动在 `where` 条件中追加 `data_status = 1`；在执行删除时会自动转变成更新 `set data_status = {主键值}`，例如：

- 删除： `update user set data_status = 12301278194792831 where id = 12301278194792831 and data_status = 1`
- 查询： `select * from user where data_status = 1`

2.5 应用案例

创建继承 `BaseDO` 的持久化类与数据库中的表进行映射，使用注解指定表名、主键字段映射属性、主键值生成策略、字段是否需要持久化、字段持久化策略等，示例如下：

```
1 @TableName(value = "t_user", autoResultMap = true)
2 public class UserDO extends BaseDO {
3
4     @TableId(type = IdType.ASSIGN_ID)
5     private Long userId;
6
7     @TableField(typeHandler = Sm4CryptoTypeHandler.class)
8     private String tel;
9
10    @TableField(typeHandler = TypeHandlerBooleanToInteger.class)
11    private Boolean enable;
12
13    @TableField(updateStrategy = FieldStrategy.IGNORED)
14    private String remark;
15
16    private Long mainDeptId;
17
18    // setter and getter...
19 }
```

三、注解

3.1 表名注解

在实体类上使用 `TableName` 注解来指定实体类对应的数据库表

3.2 主键注解

在属性上使用 `TableId` 注解来与数据库表的主键映射，并设置主键生成策略；默认使用雪花算法 `IdType.ASSIGN_ID` 该算法是 Twitter 开源的分布式 ID 生成算法，结果是一个 long 型的

Id，其核心思想是使用 41bit 作为毫秒数，10bit 作为机器的 Id，12bit 作为毫秒内的流水号，最后一个符号永远是0，因此基本可以保证全球唯一有序；如果设置成 `IdType.ASSIGN_UUID` 则会生成排除下划线的 UUID 字符串作为主键值，无法保证有序；如果设置成 `IdType.AUTO` 则数据库字段一定要设置成自增的（Mysql 可以，但 Oracle 不行）；如果不想使用数据库的自增主键；也可以使用 `[IdType.INPUT]`(<https://baomidou.com/pages/e131bd/>) 自己传递主键进行插入工作，Mybatis Plus 内置了一些主键序列（DB2KeyGenerator、H2KeyGenerator、KingbaseKeyGenerator、OracleKeyGenerator、PostgreKeyGenerator），如果内置的不能满足你的需求，可以实现 `IKeyGenerator` 接口来扩展。

3.3 自定义主键生成策略

Mybatis Plus 的 ID 生成器主要分为两类，IdentifierGenerator 和 IKeyGenerator

2.3.1 IdentifierGenerator

适用场景是不依赖数据库生成 ID，而是由 Mybatis Plus 自己提供一套 ID 生成算法，对应的主键生成方式为 `IdType.ASSIGN_ID`、`IdType.ASSIGN_UUID`，接口中包含三个方法

- `assignId`（是否需要分配 id）
- `nextId`（获取下一个数值型 id）
- `nextUUID`（获取下一个 uuid）

具体使用：

```
1 public class CustomIdGenerator implements IdentifierGenerator {
2     @Override
3     public Long nextId(Object entity) {
4         //可以将当前传入的class全类名来作为bizKey,或者提取参数来生成bizKey进行分布式Id调用生成.
5         String bizKey = entity.getClass().getName();
6         //根据bizKey调用分布式ID生成
7         long id = ....;
8         //返回生成的id值即可.
9         return id;
10    }
11 }
```

```
1 @TableName(value ="t_user")
2 public class User extends BaseDO {
3
4     @TableId(value = "user_id",type = IdType.ASSIGN_ID)
```

```

5 private Long userId;
6 private String name;
7 // setter and getter...
8 }

```

```

1 @Configuration
2 public class IdAutoConfig {
3     @Bean
4     public IdentifierGenerator idGenerator() {
5         return new CustomIdentifierGenerator();
6     }
7 }

```

3.3.2 IKeyGenerator

该生成器主要是根据不同的数据库类型，执行 SQL 语句生成对应的主键，典型的数据库如 Oracle、PostgreSQL 需要根据序列其生成主键值，可以参考 `OracleKeyGenerator` 实现：

```

1 public class OracleKeyGenerator implements IKeyGenerator {
2     public OracleKeyGenerator() {
3     }
4
5     public String executeSql(String incrementerName) {
6         return "SELECT " + incrementerName + ".NEXTVAL FROM DUAL";
7     }
8
9     public DbType dbType() {
10         return DbType.ORACLE;
11     }
12 }

```

具体使用：

- 在实体种通过 `KeySequence` 注解指定序列器名称，并通过 `TableId` 注解指定主键生成策略为 `IdType.INPUT`

```

1 @KeySequence(value = "SEQ_ORACLE_STRING_KEY", clazz = String.class)
2 public class UserDO extends BaseDO {
3
4     @TableId(value = "user_id", type = IdType.INPUT)
5     private String userId;

```

```
6
7 }
```

- spring boot 配置列中配置 keyGenerator 具体实现类

```
▼
1 @Bean
2 public IKeyGenerator keyGenerator() {
3     return new OracleKeyGenerator();
4 }
```

3.4 字段加密存储

`brick-core-orm-plus` 组件通过类型处理器实现字段数据的加密存储，加密处理器有 `Sm4CryptoTypeHandler`、`AesCryptoTypeHandler`、`DesCryptoTypeHandler`，可针对不同的数据源指定不同的密钥，如果不指定则使用默认密钥 `shinowshinow` 具体使用：

```
▼
1 @TableField(typeHandler = Sm4CryptoTypeHandler.class)
2 private String tel;
```

```
▼
1 @Configuration
2 public class FieldEncryptKeyConfig {
3
4     @Value("${encrypt-key.master.filedPrivacyKey}")
5     private String masterKey;
6     @Value("${encrypt-key.report.filedPrivacyKey}")
7     private String reportKey;
8
9     @Bean
10    public FieldEncryptKeySupplier fieldEncryptKeySupplier() {
11        return () -> {
12            Map<String, String> keyMap = new HashMap<>();
13            keyMap.put("master", masterKey);
14            keyMap.put("report", reportKey);
15            return keyMap;
16        };
17    }
18 }
```

3.5 字段类型处理器

类型处理器用于 `javaType` 与 `JdbcType` 之间的转换，如：对于数据库中字段类型为 `tinyint` 在 `java`中属性类型为 `Boolean` 组件提供了 `TypeHandlerBooleanToInteger` 处理器：

```
1 @MappedTypes(Boolean.class)
2 @MappedJdbcTypes(JdbcType.INTEGER)
3 public class TypeHandlerBooleanToInteger extends BaseTypeHandler<Boolean> {
4     @Override
5     public void setNonNullParameter(PreparedStatement ps, int i, Boolean parameter,
6         JdbcType jdbcType) throws SQLException {
7         if (parameter == null) {
8             ps.setObject(i, null);
9         } else {
10             ps.setInt(i, parameter ? 1 : 0);
11         }
12     }
13     @Override
14     public Boolean getNullableResult(ResultSet rs, String columnName) throws SQLException {
15         Object value = rs.getObject(columnName);
16         return getBoolean(value);
17     }
18     @Override
19     public Boolean getNullableResult(ResultSet rs, int columnIndex) throws SQLException {
20         Object value = rs.getObject(columnIndex);
21         return getBoolean(value);
22     }
23     @Override
24     public Boolean getNullableResult(CallableStatement cs, int columnIndex) throws
25         SQLException {
26         Object value = cs.getObject(columnIndex);
27         return getBoolean(value);
28     }
29     private Boolean getBoolean(Object value) {
30         if (value == null) {
31             return null;
32         }
33         if (value instanceof Boolean) {
34             return (Boolean) value;
35         }
36         if (value instanceof BigInteger) {
37             return ((BigInteger) value).intValue() != 0;
38         }
39         if (value instanceof BigDecimal) {
```



```

42     return ((BigDecimal) value).intValue() != 0;
43 }
44 int intValue = (int) value;
45 return intValue != 0;
46 }
47 }

```

Mybatis-Plus 为我们内置了大多数的类型处理器，可参照其实现。

3.6 更新时字段验证策略

Mybatis Plus 的 `update()` 或者 `updateById()` 来更新数据时，默认是无法将字段设置为 `null` 值的，如果需要设置为 `null` 则需要指定字段的验证策略，策略如下：

- `FieldStrategy.IGNORED`：忽略判断
- `FieldStrategy.NOT_NULL`：非 NULL 判断
- `FieldStrategy.NOT_EMPTY`：非空判断
- `FieldStrategy.DEFAULT`：追随全局配置
- `FieldStrategy.NEVER`：不加入SQL

如需将字段设置为null则血药使用 `FieldStrategy.IGNORED` 策略

```

1 @TableField(updateStrategy = FieldStrategy.IGNORED)
2 private String remark;

```

四、注意事项

4.1 字段填充

- 填充原理是直接给 `entity` 的属性设置值
- 注解则是指定该属性在对应情况下必有值，如果无值则入库会是 `null`
- `MetaObjectHandler` 提供的默认方法的策略均为：如果属性有值则不覆盖，如果填充值为 `null` 则不填充
- 字段必须声明 `TableField` 注解，属性 `fill` 选择对应策略，该声明告知 `Mybatis-Plus` 需要预留注入 `SQL` 字段

- 填充处理器 `MyMetaObjectHandler` 在 `Spring Boot` 中需要声明 `@Component` 或 `@Bean` 注入
- 使用方法 `update(T t, Wrapper updateWrapper)` 更新时 `t` 不能为空，否则自动填充失效

4.2 乐观锁注意事项

- 版本号字段支持的数据类型只有 `int``Integer``long``Long``Date``Timestamp``LocalDateTime`
- 整数类型下 `newVersion = oldVersion + 1`
- 仅支持 `updateById(entity)` 与 `update(entity, wrapper)` 方法
- 在 `update(entity, wrapper)` 方法下，`wrapper` 不能复用

4.3 逻辑删除注意事项

1. 逻辑删除是为了方便数据恢复和保护数据本身价值等等的一种方案，实际上就是删除，如果你需要频繁查出来看就不应该使用逻辑删除，而应该以一个状态字段去表示
2. 逻辑删除字段在数据库层面需要设置默认值，否则每次 `insert` 前就需要自己 `set` 值
3. 推荐使用 `deleteById(id)` 方法或 `UpdateWrapper.set("data_status", id)` 方法执行逻辑删除
4. 创建唯一索引时必须与 `data_status` 字段创建为联合唯一索引

问：为什么要将逻辑删除记录的 `data_status` 字段值设置为与主键值？[（详见说明）](#)

答：1 如果数据库中唯一索引没有与逻辑删除字段一起创建联合唯一索引，那么插入的新数据不能与已删除的数据重复；

2 如果逻辑删除值为常量，如固定为 0，即使我们创建唯一索引时与 `data_status` 字段一同创建了联合唯一索引，在被删除的记录里也不能出现重复的数据。

4.4 表名注解注意事项

**如果实体类中使用到了 `** typeHandler **` 则 `** autoResultMap` 必须设置为 `true`，否则 `typeHandler` 会失效

4.5 主键注解注意事项

注意事项：一般推荐使用默认的 `IdType.ASSIGN_ID` 策略，表中的主键字段设置为 `bigint` 类型，持久化映射类的主键属性设置为 `Long` 类型

1. 整型且有序的主键值在InnoDB存储引擎种能够有效避免数据页分裂提高数据插入的速度
2. 雪花算法能够很好的兼容分布式环境

4.6 字段加密存储注意事项

注意事项:

1. 需要将 `TableName` 注解中的 `autoResultMap` 设置为 `true` 自动加解密才会生效
2. 不支持模糊查询

4.7 字段类型处理器注意事项

注意事项: 必须开启映射注解!!! `@TableName(autoResultMap = true)`

五、数据变更记录

- 在对应的实体类上添加注解 `@LogicTable` 指定该类对应业务名称。
- 在需要被记录的字段上添加注解 `@LogicField` 指定该字段对应的业务属性。
- 当存储的字段内容是编码时, 可以在 `@LogicField` 中指定字典类型 `dictionaryType` 来解析字段的业务值, 同时还需要实现 `DataOptionService.getOptionOfCode` 方法来定义解析方式 (在接下来会有示例)。
- 自定义日志的存储逻辑时需要实现 `HandleLogicInfo` 接口。

实体类:

```
1
2 @LogicTable("角色")
3 @TableName(value = "sys_role")
4 public class RoleDO implements DataObject {
5
6     private static final long serialVersionUID = 563227379317570382L;
7     @TableId
8     private Long uuid;
9     private Long parentUuid;
10    @LogicField(value = "名称", flag = true)
11    private String name;
12    @LogicField(value = "描述")
13    private String description;
14    @LogicField(value = "启用", dictionaryType = "switch_type")
15    private Boolean enable;
```

```
16 }
```

自定义字典解析方式:

此处自定义字典获取方式（可以通过查询数据库获取，也可以从内存中获取）

```
1
2 import java.util.ArrayList;
3
4 @Service
5 public class DataOptionServiceImpl implements DataOptionService {
6
7     @Override
8     public List<OptionDTO> getOptionOfCode(String dictCode) {
9         List<OptionDTO> optionDTOList = new ArrayList<>();
10        if ("switch_type".equals(dictCode)) {
11            optionDTOList.add(new OptionDTO(true, "启用"));
12            optionDTOList.add(new OptionDTO(false, "禁用"));
13        }
14        return optionDTOList;
15    }
16 }
```

自定义数据操作日志处理方式:

本示例仅做打印处理

```
1
2 @Service
3 public class LogicInfoDefaultHandler implements HandleLogicInfo {
4     private final Logger log = LoggerFactory.getLogger(LogicInfoDefaultHandler.class);
5     private final ObjectMapper objectMapper = new ObjectMapper()
6         .setSerializationInclusion(JsonInclude.Include.NON_NULL)
7         .setSerializationInclusion(JsonInclude.Include.NON_EMPTY);
8
9     @SneakyThrows
10    @Override
11    public void insertHandler(LogicInfo logicInfo) {
12        log.info("brick-core-orm-plus insert data \n{}",
13            JSONStrFormatter.format(objectMapper.writeValueAsString(logicInfo)));
14    }
15
16    @SneakyThrows
17    @Override
18    public void updateHandler(LogicInfo logicInfo) {
```

```

18     log.info("brick-core-orm-plus update data \n{}",
JSONStrFormatter.format(objectMapper.writeValueAsString(logicInfo)));
19 }
20
21 @SneakyThrows
22 @Override
23 public void deleteHandler(LogicInfo logicInfo) {
24     log.info("brick-core-orm-plus delete data \n{}",
JSONStrFormatter.format(objectMapper.writeValueAsString(logicInfo)));
25 }
26 }

```

5.1 LogicField 注解中属性对应的含义

属性	描述
value	字段对应的业务名称
flag	是否是关键业务项
table	关联表（解决存储时使用的是标识，而展示时需要显示具体内容的问题）
column	关联列（解决存储时使用的是标识，而展示时需要显示具体内容的问题）
displayColumn	目标列（解决存储时使用的是标识，而展示时需要显示具体内容的问题）
dictionaryType	用来解析字段值的静态字典（解决存储时使用的是编码，而展示时需要显示具体内容的问题）

5.2 自定义变更日志的处理逻辑

实现 HandleLogicInfo 接口

```

▼
1
2 @Component
3 public class MyLogicInfoDefaultHandler implements HandleLogicInfo {
4     private final Logger log = LoggerFactory.getLogger(MyLogicInfoDefaultHandler.class);
5
6     @Override
7     public void insertHandler(LogicInfo logicInfo) {
8         // do something...
9     }
10
11     @Override

```

```

12 public void updateHandler(LogicInfo logicInfo) {
13     // do something...
14 }
15
16 @Override
17 public void deleteHandler(LogicInfo logicInfo) {
18     // do something...
19 }
20 }

```

变更日志功能会自动将构建完成的变更数据作为参数传递给 HandleLogicInfo 的各个方法，参数 logicInfo 数据内容如下：

insertHandler 方法中的 logicInfo 参数

```

1 {
2   "operateType": "INSERT",
3   "tableName": "sys_role",
4   "operateTime": "2021-03-12 15:45:51",
5   "content": {
6     "logicName": "角色",
7     "highlightsContent": {
8       "name": {
9         "logicName": "名称",
10        "newValue": "测试"
11      }
12    },
13    "commonContent": {
14      "description": {
15        "logicName": "描述",
16        "newValue": "测试角色"
17      },
18      "parentUuid": {
19        "logicName": "父角色",
20        "newValue": 1370252464978042881,
21        "table": "sys_role",
22        "column": "uuid",
23        "displayColumn": "name"
24      }
25    }
26  }
27 }

```

updateHandler 方法中的 logicInfo 参数

```
▼
1 {
2   "operateType": "UPDATE",
3   "tableName": "sys_role",
4   "primaryKey": 1370279857738420225,
5   "operateTime": "2021-03-12 15:47:14",
6   "content": {
7     "logicName": "角色",
8     "highlightsContent": {
9       "name": {
10         "logicName": "名称",
11         "oldValue": "测试",
12         "newValue": "开发"
13       }
14     },
15     "commonContent": {
16       "description": {
17         "logicName": "描述",
18         "oldValue": "测试角色",
19         "newValue": "开发角色"
20       },
21       "parentUuid": {
22         "logicName": "父角色",
23         "oldValue": 1370252464978042881,
24         "table": "sys_role",
25         "column": "uuid",
26         "displayColumn": "name"
27       }
28     }
29   }
30 }
```

deleteHandler 方法中的 logicInfo 参数

```
▼
1 {
2   "operateType": "DELETE",
3   "tableName": "sys_role",
4   "primaryKey": 1370279857738420225,
5   "operateTime": "2021-03-12 16:05:50",
6   "content": {
7     "logicName": "角色",
8     "highlightsContent": {
9       "name": {
10        "logicName": "名称",
11        "oldValue": "开发"
12      }
13     }
14   }
15 }
```

```
13 },
14 "commonContent": {
15     "deleted": {
16         "logicName": "启用",
17         "oldValue": false,
18         "parseDic": {
19             "true": "是",
20             "false": "否"
21         }
22     },
23     "description": {
24         "logicName": "描述",
25         "oldValue": "开发角色"
26     },
27     "parentUuid": {
28         "logicName": "父角色",
29         "oldValue": 1370252464978042881,
30         "table": "sys_role",
31         "column": "uuid",
32         "displayColumn": "name"
33     }
34 }
35 }
36 }
```

六、推荐使用方式

- 在项目中添加 project-module-admin 依赖，创建DO时直接集成开发好的 BaseDO。后端添加依赖如下：

```
▼
1 runtimeOnly "com.shinow.brick:project-module-admin:${adminVersion}"
2 implementation "com.shinow.brick:project-common-admin:${adminVersion}"
```

注意：adminVersion 截止到文档编写日期版本为 1.0.131 [查看最新版](#)